1. Execution of the original application

```
PS C:\WINDOWS\system32> cd "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7"
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7> .\.venv\Scripts\python.exe processor_original.py
==== Running parse_order ====
Enter garment:weight in kg (e.g. regular:6): regular:6
Garment and Weight: ('Regular', 6.0)

==== Running weight_average ====
Enter garment weights as CSV (e.g. 3,5,8): 3,4
Average Weight: 3

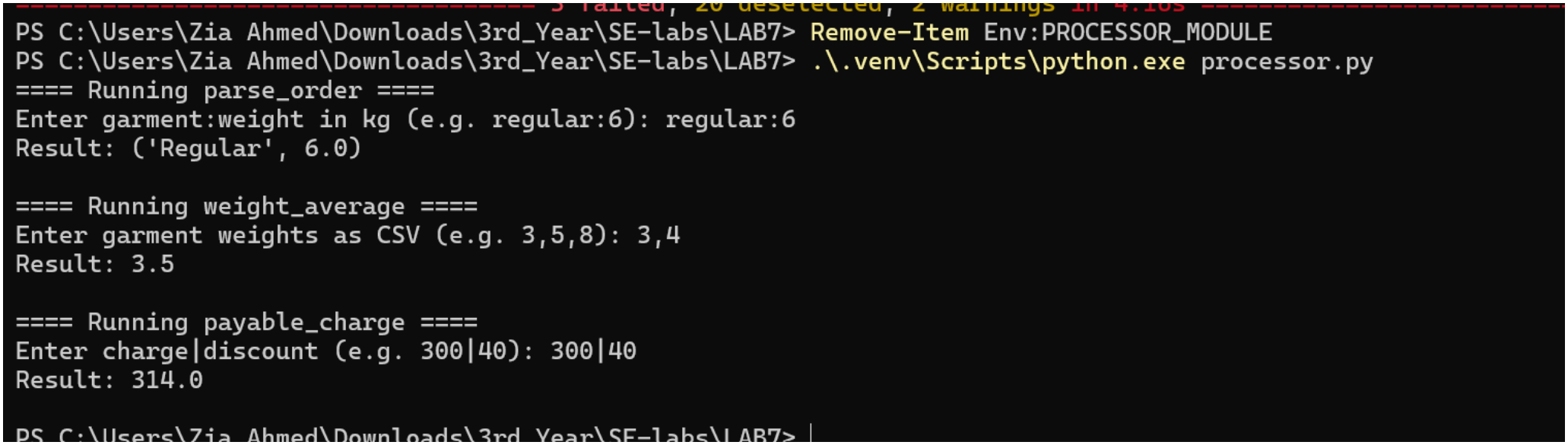
==== Running payable_charge ====
Enter charge|discount (e.g. 300|40): 300|40
Payable Charge: 314.0
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7>
```

2. Failing test cases identified during fuzz testing

```
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7> $env:PROCESSOR_MODULE = "processor_original"
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7> .\.venv\Scripts\python.exe -m pytest test_processor.py -k no_cras
h -v --tb=short
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\.venv
\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase(WindowsPath('C:/Users/Zia Ahmed/Downloads/3rd_Yea
r/SE-labs/LAB7/.hypothesis/examples'))
rootdir: C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7
plugins: hypothesis-6.112.1
collected 23 items / 20 deselected / 3 selected

test_processor.py::test_parse_order_no_crash FAILED                                                  [ 33%]
test_processor.py::test_weight_average_no_crash FAILED                                               [ 66%]
test_processor.py::test_payable_charge_no_crash FAILED                                               [100%]
```

3. Execution of the corrected application



4. Final test execution showing successful results

```
test_processor.py::test_parse_order_no_crash PASSED                                  [  4%]
test_processor.py::test_weight_average_no_crash PASSED                               [  8%]
test_processor.py::test_payable_charge_no_crash PASSED                               [ 13%]
test_processor.py::test_parse_order_valid PASSED                                     [ 17%]
test_processor.py::test_weight_average_valid PASSED                                  [ 21%]
test_processor.py::test_payable_charge_valid PASSED                                  [ 26%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-None] PASSED         [ 30%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-] PASSED             [ 34%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-regular:-1] PASSED   [ 39%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-regular:nan] PASSED  [ 43%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-regular:inf] PASSED  [ 47%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-unknown:5] PASSED    [ 52%]
test_processor.py::test_invalid_inputs_are_rejected[weight_average-None] PASSED      [ 56%]
test_processor.py::test_invalid_inputs_are_rejected[weight_average-] PASSED          [ 60%]
test_processor.py::test_invalid_inputs_are_rejected[weight_average-3,,5] PASSED      [ 65%]
test_processor.py::test_invalid_inputs_are_rejected[weight_average--1,5] PASSED      [ 69%]
test_processor.py::test_invalid_inputs_are_rejected[weight_average-nan,5] PASSED     [ 73%]
test_processor.py::test_invalid_inputs_are_rejected[payable_charge-None] PASSED      [ 78%]
test_processor.py::test_invalid_inputs_are_rejected[payable_charge-] PASSED          [ 82%]
test_processor.py::test_invalid_inputs_are_rejected[payable_charge--10|2] PASSED     [ 86%]
test_processor.py::test_invalid_inputs_are_rejected[payable_charge-300|nan] PASSED   [ 91%]
test_processor.py::test_invalid_inputs_are_rejected[payable_charge-1e309|0] PASSED   [ 95%]
test_processor.py::test_fractional_average_regression PASSED                        [100%]

============================================== 23 passed in 2.90s ==============================================
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7>
```

processor.py

Lines 1-60 of 69

```
"""Lab 7 laundry input processing. Invalid input returns None."""

import math

GARMENTS = {"regular", "delicate", "bedding"}


def _nonnegative_number(text):
    """Parse a finite, nonnegative number; return None on invalid input."""
    try:
        value = float(text)
    except (ValueError, TypeError, OverflowError):
        return None
    return value if math.isfinite(value) and value >= 0 else None


def parse_order(text):
    if not isinstance(text, str) or text.count(":") != 1:
        return None
    garment, raw_weight = text.split(":")
    garment = garment.strip().lower()
    weight = _nonnegative_number(raw_weight)
    if garment not in GARMENTS or weight is None:
        return None
    return garment.title(), weight


def weight_average(weights_csv):
    if not isinstance(weights_csv, str) or not weights_csv.strip():
        return None
    values = [_nonnegative_number(part) for part in weights_csv.split(",")]
    if any(value is None for value in values):
        return None
    # Divide first so even a list of very large finite weights can be averaged.
    try:
        average = math.fsum(value / len(values) for value in values)
    except OverflowError:
        return None
    return average if math.isfinite(average) else None


def payable_charge(text):
    if not isinstance(text, str) or text.count("|") != 1:
        return None
    raw_charge, raw_discount = text.split("|")
    charge = _nonnegative_number(raw_charge)
    discount = _nonnegative_number(raw_discount)
    if charge is None or discount is None:
        return None
    total = charge * 1.18 - discount
    if not math.isfinite(total):
        return None
    return max(total, 0.0)


def main():
    for heading, prompt, function in [
        ("parse_order", "Enter garment:weight in kg (e.g. regular:6): ", parse_order),
        ("weight_average", "Enter garment weights as CSV (e.g. 3,5,8): ", weight_average),
        ("payable_charge", "Enter charge|discount (e.g. 300|40): ", payable_charge),
```

processor.py (continued)

Lines 61-69 of 69

```
    ]:
        print(f"==== Running {heading} ====")
        result = function(input(prompt))
        print("Invalid input" if result is None else f"Result: {result}")
        print()


if __name__ == "__main__":
    main()
```

test_processor.py

Lines 1-60 of 90

```
"""Use PROCESSOR_MODULE=processor_original to reproduce original failures."""

import importlib
import math
import os

import pytest
from hypothesis import given, settings, strategies as st

app = importlib.import_module(os.environ.get("PROCESSOR_MODULE", "processor"))

arbitrary_input = st.one_of(
    st.none(), st.text(), st.booleans(), st.integers(), st.floats(),
    st.lists(st.integers(), max_size=5),
    st.dictionaries(st.text(max_size=10), st.integers(), max_size=3),
)
weights = st.floats(min_value=0, max_value=1_000_000, allow_nan=False, allow_infinity=False)


@settings(max_examples=300)
@given(arbitrary_input)
def test_parse_order_no_crash(value):
    result = app.parse_order(value)
    if result is not None:
        garment, weight = result
        assert garment in {"Regular", "Delicate", "Bedding"}
        assert math.isfinite(weight) and weight >= 0


@settings(max_examples=300)
@given(arbitrary_input)
def test_weight_average_no_crash(value):
    result = app.weight_average(value)
    assert result is None or (math.isfinite(result) and result >= 0)


@settings(max_examples=300)
@given(arbitrary_input)
def test_payable_charge_no_crash(value):
    result = app.payable_charge(value)
    assert result is None or (math.isfinite(result) and result >= 0)


@given(st.sampled_from(["regular", "delicate", "bedding"]), weights)
def test_parse_order_valid(garment, weight):
    assert app.parse_order(f" {garment.upper()} : {weight} ") == (garment.title(), weight)


@given(st.lists(weights, min_size=1, max_size=30))
def test_weight_average_valid(values):
    result = app.weight_average(",".join(map(str, values)))
    assert result == pytest.approx(sum(values) / len(values))
    assert min(values) <= result + 1e-8
    assert result <= max(values) + 1e-8


@given(weights, weights)
def test_payable_charge_valid(charge, discount):
    assert app.payable_charge(f"{charge}|{discount}") == pytest.approx(
        max(charge * 1.18 - discount, 0)
```

test_processor.py (continued)

Lines 61-90 of 90

```
    )


@pytest.mark.parametrize(
    "function,value",
    [
        ("parse_order", None),
        ("parse_order", ""),
        ("parse_order", "regular:-1"),
        ("parse_order", "regular:nan"),
        ("parse_order", "regular:inf"),
        ("parse_order", "unknown:5"),
        ("weight_average", None),
        ("weight_average", ""),
        ("weight_average", "3,,5"),
        ("weight_average", "-1,5"),
        ("weight_average", "nan,5"),
        ("payable_charge", None),
        ("payable_charge", ""),
        ("payable_charge", "-10|2"),
        ("payable_charge", "300|nan"),
        ("payable_charge", "1e309|0"),
    ],
)
def test_invalid_inputs_are_rejected(function, value):
    assert getattr(app, function)(value) is None


def test_fractional_average_regression():
    assert app.weight_average("3,4") == 3.5
```